

Embedded Take-Home: Real-Time AGC

Time: aim for 6–8 hours. Do as much as you can in that time. This is a broad task and we do not expect everyone to finish all of it — see the note at the end.

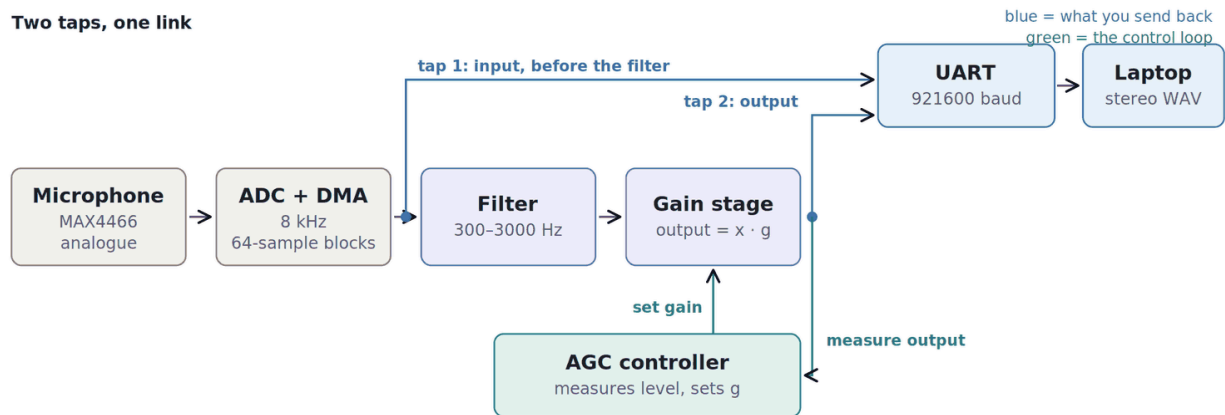
Hardware: NUCLEO-G474RE, MAX4466 microphone module, jumper wires, USB cable. Keep it set up after submitting; we will use it on the follow-up call.

1. What you are building

A small real-time audio system on the STM32, running bare metal.

Sound comes in through the microphone. You filter it, then pass it through a gain stage. A control loop watches how loud the output is and adjusts that gain to keep the loudness roughly constant — quiet sound gets boosted, loud sound gets turned down. That is an automatic gain control, or AGC. It is the thing that keeps someone's voice at a steady level on a video call when they lean towards or away from their laptop.

You send two signals back to your laptop over USB: the audio as it arrived, and the audio after processing. We compare them.



Everything left of the laptop runs bare metal on the STM32.

Figure 1 — The system. Two taps go back to your laptop; the control loop is in green.

2. What we give you

A small library, so you do not spend the day on plumbing:

- `framing.c` — packs your samples into frames with a checksum, ready to send.
- `ringbuf.c` — a buffer for holding frames while the UART drains them.
- `instrument.h` — a cycle counter and a set of counters for spotting dropped data. You do not need to write any measurement code; just call these.
- `host_receive.py` — run this on your laptop. It receives the stream and writes a stereo WAV plus a CSV. It also prints your sample rate, any gaps, and your timing numbers.

- A test audio track. Play it into the microphone from your phone or a speaker.

Run `make test` in the library folder before you start. It should pass in a few seconds. That confirms the framing and the host script agree, so the link is one less thing to worry about.

3. What to build

3.1 Capture

- Sample the microphone at 8 kHz using a hardware timer to trigger the ADC. Do not sample in a software loop.
- It has to run indefinitely without dropping samples.

3.2 Filter

Remove the DC offset — the microphone module sits at about half the supply voltage, and if you leave that in, every level measurement you make will be wrong.

Then keep the 300–3000 Hz band. The parts of the test track we measure all sit inside that range, but the track also carries tones outside it — low rumble and a high tone near the top of the audio range — which your filter needs to remove. **The type and order are up to you**, and generated or copied coefficients are completely fine. We care that you can explain what your filter does to the signal.

There is also a slow frequency sweep in the track that runs from below the band to above it. Your recording of it will show the shape of your filter directly, which is a good way to check your coefficients are doing what you think.

3.3 AGC

Hold the output at roughly -20 dBFS RMS. In outline:

- Measure how loud your output currently is.
- Compare that to the target.
- Nudge the gain up or down to close the gap.
- Repeat, every block.

Move the gain down quickly and back up slowly. Something like 10 ms to reduce gain and 150 ms to restore it works well — use those numbers, or your own if you prefer. There is no hidden tuning target here.

Two things to get right:

- **Clamp the gain** to somewhere between -20 dB and $+40$ dB, so it cannot run away during silence.
- **Change the gain smoothly**, not in one jump per block, or you will hear it stepping.

When the input suddenly gets much louder, the output will jump before the loop catches up. That is unavoidable and expected. What matters is that it settles back cleanly rather than oscillating, and that your output saturates rather than wrapping around when it hits full scale.

3.4 Send it back

Stream the input and the output to your laptop, interleaved, one frame per block, using the library. Add a small status record 20 times a second carrying your gain, your levels, your worst block time and the counters. `host_receive.py` turns all of that into a WAV and a CSV.

The one real constraint: the audio must never miss a block because the sending got in the way. Roughly 334 kbps has to keep flowing while capture continues. You choose the buffer size and what happens if your laptop falls behind — if it does, drop stream data and count it rather than stalling the audio.

If the link cannot keep up. Some ST-LINK revisions will not hold 921600 baud. Do not quietly drop data: either find a rate that works, or halve the sample rate to 4 kHz and move your filter down accordingly. Either way, say what you did. Working around a hardware limit deliberately is a good answer.

3.5 Recording the track

Play the track from your phone or a laptop, twenty to thirty centimetres from the microphone, in a reasonably quiet room.

The track opens with about six seconds of a steady tone at the loudest level it ever reaches. Use that to set the microphone module's trim pot. Run:

```
python host/host_receive.py --port /dev/ttyACM0 --monitor
```

and adjust the pot until the peak sits a few dB below full scale — loud, but never touching 0 dBFS. Then **leave the pot alone** for the rest of the exercise. Adjusting the analogue gain later is not a substitute for the AGC, and it makes your recordings hard to compare.

Then record the whole track in one go, about two minutes.

3.6 Ground rules

- **Bare metal.** No RTOS.
- **No dynamic allocation.** No `malloc` and friends. All memory static or on the stack.
- Use the HAL or CubeMX for setup if you like. Your DMA, timing and DSP paths should be code you understand and can talk through.

4. Measuring

You should not need an oscilloscope or any other test equipment. The helpers in the library cover it:

- **Block timing.** Wrap your processing in `BLOCK_TIMER_START()` and `BLOCK_TIMER_END()`. That records your worst-case time in CPU cycles. Report the worst case, not the average — the average hides the one block that missed its deadline.
- **Dropped data.** Increment the counters in `g_counters` when a block arrives while you are still busy, when the ADC reports an overrun, or when your buffer is full. All of them should read zero at the end of a run.
- **Your clock speed.** Tell us what you configured and how you checked. Your time budget per block is your clock times 8 ms, so the cycle numbers mean nothing without it.

- **Sample rate.** `host_receive.py` measures this against your laptop's clock and prints it. Nothing for you to do.

5. What to send back

A git repository, with normal incremental commits, containing:

1. **Your firmware**, building from documented steps.
2. `stereo_capture.wav` and `telemetry.csv` — recorded while playing the full test track (about two minutes) into the microphone. The host script writes both.
3. A plot of your gain and output level over time, from the CSV. This is the most useful single thing you can send us.
4. The filled-in report template, including the short note on AI use.

6. On AI assistance

Use AI tools if you want to. We do. We are not testing whether you can avoid them.

We are testing whether you understand what you shipped, so tell us which parts you used them for and — more usefully — what they got wrong that you had to fix. A specific answer there tells us more than a clean codebase does. Saying you used none scores no higher.

7. The follow-up call

Around 45 minutes, with the board connected and your code running. We will ask you to explain your design and talk through the decisions you made. You may also be asked to make a small change and predict what it will do, so it is worth having your development environment ready.

8. A few tips

- A quick way to test without the track: feed your processing chain a synthetic tone that jumps in volume every half second, instead of the ADC. Easier to reason about than a live microphone, and worth twenty minutes.
- Fill in the report as you go. The numbers are easier to record when you first measure them.

Before you start

This is a broad challenge, and it covers a lot of ground on purpose so there is something here for a range of backgrounds.

Do as much of it as you can in the time, and do not worry if you do not finish. A smaller piece that works, with honest notes on what you measured and what you would do next, is worth more to us than everything half-finished. If something is not working, tell us what you tried.

AI use is encouraged. Use whatever tools you normally would. We only ask that you can explain what you shipped, and that you tell us what the tools got wrong.

There may be mistakes in this document or in the code we have given you. If something looks wrong or contradictory, use your judgement, do whatever seems sensible, and note the assumption in your report. Spotting a problem and saying so is a good outcome, not a failed task.